
FinWise AI Project Report

Abstract

FinWise AI is a local personal finance management system that helps a user track bank accounts, cash, UPI wallets, credit cards, loans, income, expenses, transfers, budgets, savings goals, analytics, reports, statement imports, and finance guidance. The project is designed as an offline-first application, so the main finance features and the built-in AI assistant can work without an internet connection after the required software is installed.

The system uses a React frontend, Flask backend, MySQL database, and explainable finance algorithms. The AI part is not a cloud chatbot. It uses local rules, user transaction history, and a Multinomial Naive Bayes category prediction model to answer finance-related questions and help classify transactions.

Problem Statement

Many students and individuals record money details manually or keep them spread across bank apps, notes, and spreadsheets. This makes it difficult to know:

- How much money is currently available.
- Where money is being spent.
- How much was earned and spent in a selected month.
- Whether a budget is being crossed.
- How much has been saved toward goals.
- How loans, credit cards, and transfers affect account balances.

FinWise AI solves this by providing one simple finance dashboard with transaction tracking, account-wise ledger, analytics, statement import, and offline finance guidance.

Objectives

- Track total money, balance money, income, expenses, savings, transfers, loans, and credit cards.
- Maintain multiple account types such as bank, cash, UPI wallet, credit card, loan, and investment.
- Import bank statements from CSV, Excel, and text-based PDF files.
- Detect duplicate imported transactions before saving.
- Provide month-wise analytics and category-wise expense breakdown.
- Maintain a ledger with debit, credit, and running balance.
- Support budgets and savings goals.
- Provide an offline finance chat assistant for project and finance questions.
- Use simple, explainable algorithms suitable for an academic project.

Scope

The project focuses on personal finance tracking for one user account at a time. It supports local database storage and local financial analysis. It does not connect directly to real bank APIs, does not send real emails without provider setup, and does not require a paid online AI service.

Technology Stack

Frontend

- React for the user interface.
- Vite for frontend build.
- Bootstrap and Bootstrap Icons for layout and icons.
- Chart.js for charts and analytics.

Backend

- Flask for REST API services.
- MySQL Connector for database communication.
- JWT and httpOnly cookies for login sessions.
- Werkzeug password hashing for secure password storage.
- openpyxl and xlrd for Excel statement files.
- pdfplumber for text-based PDF bank statements.
- reportlab for report generation support.

Database

- MySQL stores users, accounts, transactions, budgets, savings goals, goal contributions, statement import history, preferences, reports, and automation records.

Main Modules

1. Authentication and Profile

The system supports registration, login, logout, profile viewing, profile editing, password change, and theme preferences. Passwords are hashed before being stored. Sessions use httpOnly cookies, which is safer than storing login tokens directly in browser localStorage.

2. Dashboard

The dashboard shows the most important finance summary:

- Total Money: balance money plus saved goal money.
- Balance Money: available amount in accounts.
- Income: money received.
- Expense: money spent.
- Savings: money saved in goals.

It also shows monthly overview, recent transactions, budget risk, and quick actions.

3. Accounts

The accounts module supports different account types:

- Bank
- Cash

-
- UPI Wallet
 - Credit Card
 - Loan
 - Investment

This is useful because a user can track savings accounts, loan balances, and credit card balances in the same project while still keeping them separated by account type.

4. Transactions

Transactions can be added manually as:

- Income
- Expense
- Transfer

Income increases an account balance. Expense reduces an account balance. Transfer moves money from one account to another without counting it as income or expense.

5. Bank Statement Import

The statement import module allows the user to upload CSV, Excel, or text-based PDF statement files. The system previews rows before importing. The user can correct date, description, type, category, and amount before saving.

The import feature also checks possible duplicate rows so the same bank statement does not create repeated transactions.

6. Ledger

The ledger module gives an account-wise debit and credit view. It calculates a running balance for each account.

Ledger rule:

$$\text{Running Balance} = \text{Opening Balance} + \text{Credits} - \text{Debits}$$

For a transfer, the ledger creates two movements:

- Debit from the source account.
- Credit to the destination account.

This makes the project stronger because it uses accounting-style transaction tracking.

7. Budgets

The budget module lets the user set category-wise spending limits. The system compares actual expense against the budget limit and can show when spending is close to or above the limit.

8. Savings Goals

Savings goals help the user move money from an account into a goal. This is not treated as a normal expense because the money is still owned by the user. It is tracked separately as savings.

Example:

- Moving Rs 1000 to a phone savings goal reduces available account balance.
- It increases saved goal money.
- It does not count as food, bills, rent, or other expense.

9. Analytics

Analytics gives selected-month checking. The user can compare income, expense, net flow, saved-to-goal money, top category, category chart, and transactions for a selected month.

10. Reports

Reports help export or view summarized financial data. This is useful for checking monthly progress and explaining finance history.

11. Smart Tools

Smart Tools includes auto entries for repeated income or expenses such as salary, rent, subscriptions, EMI, and bills. This avoids entering the same transaction manually every month.

12. Offline Finance Chat

The chat assistant answers finance and project-related questions locally. It can explain:

- Total money, balance money, income, expense, and savings.
- Transfers between accounts.
- Ledger and running balance.
- Algorithms used in the project.
- How to use accounts, loans, credit cards, budgets, and goals.
- Whether the app works offline.

The chat does not require internet or an online AI API.

Algorithms Used

1. Multinomial Naive Bayes Category Prediction

This algorithm predicts a transaction category from the transaction description. It uses finance keywords and the user's previously saved transactions.

Example:

```
"swiggy dinner" -> Food
"salary july" -> Salary
"electricity bill" -> Bills
```

Basic idea:

Predicted Category = category with highest $P(\text{category} \mid \text{words})$

The algorithm calculates:

$P(\text{category} \mid \text{words})$ proportional to $P(\text{category}) * P(\text{word1} \mid \text{category}) * P(\text{word2} \mid \text{category}) \dots$

This is explainable and suitable for a simple AI-based academic project.

2. Rule-Based Offline Chat Intent Matching

The finance chat checks the user's question for important keywords and matches it to local help topics and finance rules.

Examples:

- "is this offline" -> explains offline working.
- "what algorithm used" -> explains Naive Bayes, ledger, duplicate detection, and analytics formulas.

-
- "transfer between accounts" -> explains Transfer transaction type.
 - "what is balance money" -> explains available account balance.

This is simple, fast, offline, and explainable.

3. Ledger Running Balance Algorithm

The ledger algorithm orders account movements by date and calculates the balance after every transaction.

```
opening = account opening balance
for each ledger row:
    running_balance = running_balance + credit - debit
```

This makes it easy to verify how each transaction changed the account balance.

4. Duplicate Transaction Detection

During statement import, the system checks whether a similar transaction already exists. It compares important fields such as account, date, amount, description, and transaction type. Duplicate rows are marked before import so the user does not accidentally import the same statement again.

5. Budget Utilization Algorithm

The system compares expense against budget:

```
Budget Usage Percent = (Actual Expense / Budget Limit) * 100
```

This helps identify categories where the user is overspending.

6. Savings Rate Calculation

The system calculates savings performance using income and savings/remaining money.

```
Savings Rate = ((Income - Expense) / Income) * 100
```

This helps the user understand whether spending is controlled.

7. Goal Progress Algorithm

Each savings goal has a target and saved amount.

```
Goal Progress Percent = (Saved Amount / Target Amount) * 100
```

This tells how close the user is to completing a financial goal.

8. Month-Wise Analytics Algorithm

The analytics page filters transactions by selected month and groups them by type and category.

It calculates:

- Monthly income.
- Monthly expense.
- Net flow.
- Top expense category.
- Goal savings movement.
- Category chart values.

9. Financial Health Rules

The project can judge finance condition using simple rules such as:

-
- Spending is high if expenses are close to or above income.
 - Budget risk is high if category spending crosses the limit.
 - Savings progress is good if goal savings are increasing.
 - Available balance should be checked separately from total tracked money.

These rules make the project explainable to a guide.

Database Overview

Important database tables include:

- users: stores registered user details and hashed passwords.
- accounts: stores bank, cash, UPI, loan, credit card, and investment accounts.
- transactions: stores income, expense, and transfer records.
- budgets: stores category-wise monthly limits.
- savings_goals: stores goals and targets.
- goal_contributions: stores money moved into or out of goals.
- statement_imports: stores uploaded statement import history.
- recurring_transactions: stores auto-entry rules.
- user_preferences: stores theme and display preferences.
- password_reset_tokens: stores hashed reset tokens.
- email_verification_tokens: stores hashed email verification tokens.

Some future-ready tables may exist for advanced features, but the main user interface focuses on simple personal finance tracking.

Data Flow

```
User action in React UI
    |
    v
Flask API receives request
    |
    v
Authentication checks current user
    |
    v
Business logic applies finance rules
    |
    v
MySQL stores or reads data
    |
    v
React UI displays dashboard, tables, charts, and chat answer
```

Security Features

- Passwords are hashed using Werkzeug.
- Login uses httpOnly cookies.
- Reset and verification tokens are hidden by default.

-
- Local secrets should be stored in ``.env``.
 - User data is filtered by logged-in user ID.
 - Statement import preview lets the user review data before saving.

Offline Working

FinWise AI can run locally on the same computer using:

- Local React build.
- Local Flask server.
- Local MySQL database.
- Local rule-based finance chat.
- Local Naive Bayes category prediction.

Internet is only needed for first-time installation or if new packages must be downloaded. After setup, the project can run without internet.

Limitations

- It does not directly connect to real bank APIs.
- PDF import works best for text-based PDFs, not scanned image PDFs.
- Email invite and email reset delivery need a real email provider if used.
- The AI assistant is explainable and offline, but it is not a large online AI model.
- Mobile app support is not included.

Future Enhancements

- Bank API integration for automatic bank sync.
- OCR support for scanned PDF statements.
- Mobile application version.
- Advanced forecasting using linear regression or time series models.
- More detailed loan EMI and credit card interest calculations.
- Optional cloud AI integration for deeper natural language answers.

Testing and Validation

The project includes safety tests for important behavior:

- Weak default secrets are avoided.
- Reset tokens are hidden by default.
- Browser popup calls are avoided in frontend pages.
- Statement undo does not bulk delete older transactions.
- Frontend dependencies are pinned.
- Offline chat includes project help topics.

-
- Family option is removed from frontend.
 - Ledger feature is connected to backend and frontend.

The frontend build also validates that the React interface compiles successfully.

How to Run

Open the project folder:

```
cd C:\FinWise_AI
```

Start the application:

```
.\run.cmd
```

Open:

```
http://127.0.0.1:5000
```

Keep the run window open while using the app.

Conclusion

FinWise AI is a simple but complete personal finance management system. It tracks real financial activities such as income, expenses, transfers, loans, credit cards, budgets, goals, and account ledger. The project includes offline AI features using explainable algorithms, which makes it suitable for academic demonstration. The system is practical, understandable, and can be extended with more advanced financial algorithms in the future.

FinWise AI Viva Notes

1. What is FinWise AI?

FinWise AI is a personal finance management system. It helps users track accounts, income, expenses, transfers, budgets, savings goals, statement imports, ledger entries, analytics, reports, and finance guidance.

2. Why is it called AI?

It has offline AI-style features:

- It predicts transaction category using Multinomial Naive Bayes.
- It answers finance questions using local rule-based intent matching.
- It gives finance insights from the user's own transaction data.

3. Does it need internet?

No. After setup, it can run locally without internet. The frontend, backend, database, finance chat, and category prediction all work on the local machine.

4. What algorithms are used?

- Multinomial Naive Bayes for transaction category prediction.
- Rule-based intent matching for offline finance chat.
- Ledger running balance algorithm.
- Duplicate transaction detection during statement import.
- Budget utilization calculation.
- Savings rate calculation.
- Goal progress calculation.
- Month-wise analytics grouping.
- Financial health rules.

5. Explain Multinomial Naive Bayes in this project.

The system reads a transaction description and predicts its category.

Example:

```
"salary july" -> Salary  
"swiggy dinner" -> Food  
"electricity bill" -> Bills
```

It calculates which category is most likely based on words in the description and previous transaction data.

6. Explain the ledger algorithm.

The ledger shows debit, credit, and running balance for each account.

$\text{Running Balance} = \text{Opening Balance} + \text{Credits} - \text{Debits}$

Income is a credit. Expense is a debit. Transfer creates a debit in one account and a credit in another account.

7. What is Balance Money?

Balance Money means the money currently available in accounts. It is the amount the user can actually use now.

8. What is Total Money?

Total Money means balance money plus money saved in goals.

$$\text{Total Money} = \text{Balance Money} + \text{Goal Savings}$$

9. Why are savings goals not shown as expenses?

Goal savings are not normal expenses because the user still owns the money. The money is only moved from an account into a savings goal. So it is tracked separately as savings.

10. How are transfers handled?

Transfers are used when money moves from one account to another account owned by the same user. A transfer is not income and not expense. It only changes account balances.

11. How should loans and credit cards be added?

Loans and credit cards can be added as account types. This keeps them visible with other accounts and makes tracking easier.

12. What is the use of bank statement import?

It saves time. The user can upload CSV, Excel, or text-based PDF statements, preview rows, fix categories, and import them into transactions.

13. How does duplicate import prevention work?

Before importing, the system checks if a similar transaction already exists for the same account, date, amount, type, and description. Duplicate rows are marked so the user does not import them again.

14. What is the use of Smart Tools?

Smart Tools are for repeated income or expense entries, like salary, rent, EMI, subscriptions, and bills. It helps avoid entering the same transaction manually every time.

15. Why was Family removed?

The project is kept simple and focused. The main purpose is personal finance tracking, so removing Family makes the system easier to understand and demonstrate.

16. What security is used?

- Password hashing.
- httpOnly cookie sessions.

-
- User-specific data filtering.
 - Hidden reset tokens by default.
 - Local environment variables for secrets.

17. What are the main modules?

- Login and Profile.
- Dashboard.
- Accounts.
- Transactions.
- Ledger.
- Budgets.
- Savings Goals.
- Analytics.
- Reports.
- Finance Chat.
- Smart Tools.

18. What is the database used?

MySQL is used to store users, accounts, transactions, budgets, goals, statement imports, automation entries, and preferences.

19. What is the limitation of the AI chat?

It is offline and explainable, but it is not a large online AI model. It answers from local project rules and finance data.

20. How do you run the project?

Open the project folder and run:

```
cd C:\FinWise_AI
.\run.cmd
```

Then open:

```
http://127.0.0.1:5000
```

Keep the run window open.

FinWise AI Guide and Interview Questions

This file contains common questions a project guide, external examiner, or interviewer may ask about FinWise AI. The answers are written in simple language so they can be used directly for viva preparation.

A. Basic Project Questions

1. What is your project?

FinWise AI is a personal finance management system. It helps users track accounts, income, expenses, transfers, budgets, savings goals, ledger entries, analytics, reports, bank statement imports, and offline finance guidance.

2. Why did you choose this project?

I chose this project because personal money tracking is a real daily problem. Many people do not know where their money is going, how much they saved, or whether they are overspending. This project solves that in a simple and practical way.

3. What problem does your project solve?

It solves the problem of scattered financial records. Instead of checking many bank apps, notes, and spreadsheets, the user can manage income, expenses, accounts, loans, credit cards, goals, and monthly analytics in one place.

4. Who can use this project?

Students, working people, small families, freelancers, and anyone who wants to track personal money can use this project.

5. What is the main objective?

The main objective is to give a simple offline personal finance system that tracks money clearly and gives explainable AI-based finance help.

6. Is this a real-time finance application?

It is not connected to live bank APIs. It is a local finance tracker where the user enters data manually or imports bank statements.

7. Is it a web application?

Yes. It is a local web application. The frontend runs in the browser, the backend runs using Flask, and data is stored in MySQL.

8. Can it run offline?

Yes. After installation, the project can run offline on the same computer. It does not need an online AI API for the finance chat.

B. Features Questions

9. What are the main modules?

The main modules are Login, Profile, Dashboard, Accounts, Transactions, Ledger, Budgets, Savings Goals, Analytics, Reports, Finance Chat, Bank Statement Import, and Smart Tools.

10. What does the dashboard show?

The dashboard shows Total Money, Balance Money, Income, Expense, Savings, monthly overview, recent transactions, and quick actions.

11. What is Total Money?

Total Money means balance money plus money saved in goals.

$\text{Total Money} = \text{Balance Money} + \text{Goal Savings}$

12. What is Balance Money?

Balance Money is the money currently available in accounts.

13. What is Savings in your project?

Savings means money moved into savings goals. It is tracked separately from normal expenses.

14. Why is goal saving not counted as an expense?

Because the user still owns the money. It is only moved from a bank account into a savings goal. It is not spent on food, rent, bills, or shopping.

15. What account types are supported?

Bank, Cash, UPI Wallet, Credit Card, Loan, and Investment.

16. How do you track a loan?

A loan can be added as a Loan account. This makes the liability visible in the same finance system.

17. How do you track a credit card?

A credit card can be added as a Credit Card account. Credit card spending can be tracked separately from normal bank accounts.

18. What are the transaction types?

Income, Expense, and Transfer.

19. What is a transfer?

A transfer is money moved from one account owned by the user to another account owned by the same user. It is not income and not expense.

20. Why do you need transfer?

Without transfer, moving money between two own accounts may be wrongly counted as income or expense. Transfer keeps tracking correct.

C. Technical Stack Questions

21. Which frontend technology is used?

React is used for the frontend. Vite is used for building the frontend.

22. Which backend technology is used?

Flask is used for the backend API.

23. Which database is used?

MySQL is used to store user, account, transaction, budget, goal, report, import, and preference data.

24. Why did you use React?

React makes it easier to build reusable components and dynamic pages like dashboard, charts, forms, and tables.

25. Why did you use Flask?

Flask is simple, lightweight, and suitable for building REST APIs for a project like this.

26. Why did you use MySQL?

MySQL is reliable for structured data. Finance records have clear tables and relationships, so MySQL is a good fit.

27. What is the role of Chart.js?

Chart.js is used to show analytics charts such as category-wise expense breakdown.

28. What libraries are used for statement import?

openpyxl and xlrd are used for Excel files. pdfplumber is used for text-based PDF files. CSV files are handled using normal CSV parsing.

D. AI and Algorithm Questions

29. Where is AI used in your project?

AI is used in transaction category prediction and offline finance chat guidance.

30. What algorithm is used for category prediction?

Multinomial Naive Bayes is used for category prediction.

31. Why did you choose Naive Bayes?

Naive Bayes is simple, fast, explainable, and works well for text classification problems like predicting category from transaction descriptions.

32. How does Naive Bayes work here?

It reads words in the transaction description and calculates which category is most likely based on previous examples and finance keywords.

33. Give an example of category prediction.

```
"salary july" -> Salary  
"swiggy dinner" -> Food  
"electricity bill" -> Bills
```

34. What is the formula idea behind Naive Bayes?

$P(\text{category} \mid \text{words}) \propto P(\text{category}) * P(\text{word1} \mid \text{category}) * P(\text{word2} \mid \text{category})$

The category with the highest probability is selected.

35. Is the AI chat an online chatbot?

No. It is an offline rule-based finance assistant. It does not call ChatGPT or any cloud AI API.

36. Why is offline AI useful?

It keeps the project simple, private, and free to run. It also avoids dependency on internet and paid APIs.

37. What is rule-based intent matching?

It means the system checks keywords in the user's question and returns a matching answer from local finance rules and project help topics.

38. What algorithms are used apart from Naive Bayes?

Ledger running balance, duplicate detection, budget utilization, savings rate, goal progress, month-wise analytics, and financial health rules.

39. Explain the ledger algorithm.

The ledger starts with opening balance and updates balance after each debit or credit.

$$\text{Running Balance} = \text{Opening Balance} + \text{Credits} - \text{Debits}$$

40. Explain duplicate detection.

Before importing a statement, the system checks whether a similar transaction already exists for the same account, date, amount, type, and description.

41. Explain budget utilization.

$$\text{Budget Usage Percent} = \text{Actual Expense} / \text{Budget Limit} * 100$$

It shows how much of the budget is already used.

42. Explain goal progress.

$$\text{Goal Progress Percent} = \text{Saved Amount} / \text{Target Amount} * 100$$

It shows how close the user is to the target.

43. Explain savings rate.

$$\text{Savings Rate} = (\text{Income} - \text{Expense}) / \text{Income} * 100$$

It shows how much of income remains after expenses.

44. Explain month-wise analytics.

The system filters transactions by selected month and calculates income, expense, net flow, saved-to-goal amount, top category, and chart values.

45. What makes the algorithms explainable?

The calculations are simple and can be shown using formulas. The user and guide can understand how each result is produced.

E. Database Questions

46. What are the main database tables?

The main tables include users, accounts, transactions, budgets, savings_goals, goal_contributions, statement_imports, recurring_transactions, user_preferences, password_reset_tokens, and email_verification_tokens.

47. What is stored in the users table?

User name, email, hashed password, verification status, and login-related fields.

48. What is stored in the accounts table?

Account name, account type, subtype, balance, bank details, credit limit, interest rate, and related account information.

49. What is stored in the transactions table?

Transaction type, category, amount, description, date, account ID, transfer account ID, and import reference.

50. What is stored in savings_goals?

Goal name, target amount, saved amount, target date, and user ID.

51. Why do you need goal_contributions?

It stores each deposit or withdrawal from a savings goal, so the history of goal movement is clear.

52. Why do you need statement_imports?

It stores import history, file name, account, number of rows, and imported transaction details.

53. What is a foreign key?

A foreign key links one table to another. For example, transactions are linked to a user and an account.

54. How do you prevent one user from seeing another user's data?

Most queries filter data using the logged-in user's ID.

55. Why is MySQL suitable here?

The project has structured records and relationships, so relational tables are useful.

F. Security Questions

56. Are passwords stored directly?

No. Passwords are stored as hashes.

57. What is password hashing?

Hashing converts the password into a one-way encrypted-looking value. The original password is not stored.

58. What is used for login session?

The app uses JWT with httpOnly cookies.

59. Why are httpOnly cookies useful?

They reduce the risk of JavaScript stealing login tokens from the browser.

60. Are reset tokens visible?

No. Reset and verification tokens are hidden by default.

61. Where should database passwords be stored?

In a local `.env` file, not directly in source code.

62. Is this ready for public deployment?

It is mainly a local academic project. For public deployment, stronger production configuration, HTTPS, email provider setup, and security review are needed.

G. Statement Import Questions

63. What file formats can be imported?

CSV, Excel, and text-based PDF files.

64. What happens before import?

The system shows a preview. The user can check and correct rows before saving.

65. Why is preview important?

Bank statements may have different formats, so preview lets the user correct mistakes before data enters the database.

66. Can scanned PDF statements be imported?

Not reliably. The current PDF import is for text-based PDFs. Scanned PDFs need OCR support.

67. What if the same statement is imported twice?

Duplicate detection marks already existing transactions so repeated import can be avoided.

68. Can imported rows be undone?

The app keeps statement import history, which helps manage imported rows.

H. Ledger and Accounting Questions

69. What is a ledger?

A ledger is an account-wise record of money movement using debit, credit, and running balance.

70. Why did you add ledger?

Ledger makes the project stronger because it follows accounting-style tracking and clearly shows how each transaction changes balance.

71. What is debit?

In this project, debit means money going out of an account.

72. What is credit?

In this project, credit means money coming into an account.

73. How is income shown in ledger?

Income is shown as a credit.

74. How is expense shown in ledger?

Expense is shown as a debit.

75. How is transfer shown in ledger?

Transfer is shown as debit from the source account and credit to the destination account.

76. Why is ledger useful for debugging finance data?

It shows every account movement step by step, so wrong balances are easier to find.

I. Analytics and Finance Questions

77. What does analytics show?

Analytics shows monthly income, expense, net flow, saved-to-goal amount, top category, charts, and month transactions.

78. How can the user check last month and this month?

The Analytics page has a month selector. The user can choose the required month and compare totals.

79. What is net flow?

$\text{Net Flow} = \text{Income} - \text{Expense}$

80. What is top category?

Top category is the category where the user spent the highest amount in the selected month.

81. Why are charts useful?

Charts make spending patterns easier to understand than only reading table rows.

82. What is budget risk?

Budget risk means spending in a category is close to or above the budget limit.

83. What is financial health?

Financial health means whether income, expense, savings, budget, and balance are in a good condition.

J. Smart Tools and Automation Questions

84. What is automation in this project?

Automation means saved auto entries for repeated transactions like salary, rent, EMI, bills, and subscriptions.

85. Why is auto entry useful?

It saves time because the user does not need to manually enter the same repeated transaction again and again.

86. Give examples of auto entries.

Salary, rent, electricity bill, EMI, internet bill, subscription, and insurance payment.

87. Is automation the same as bank sync?

No. Automation is local recurring entry setup. Bank sync would need real bank API integration.

K. Frontend Questions

88. How is navigation handled?

The dashboard has sidebar links to pages such as Accounts, Transactions, Ledger, Budget, Goals, Analytics, Reports, Finance Chat, Smart Tools, and Profile.

89. How is data displayed?

Data is displayed using cards, forms, tables, charts, and summary sections.

90. How is dark mode handled?

Theme preference is available in the profile area. The user can choose light, dark, or system mode.

91. Why use cards in dashboard?

Cards make important values like income, expense, and balance easy to see quickly.

92. Why use tables?

Tables are useful for transactions, ledger entries, imports, budgets, and history because they show structured records clearly.

L. Backend Questions

93. What is the role of Flask API?

The Flask API receives requests from the frontend, checks login, applies finance logic, reads or writes MySQL data, and sends JSON responses.

94. What is JSON?

JSON is a common data format used to send data between frontend and backend.

95. What is an API route?

An API route is a backend URL that performs a specific action, such as loading accounts or saving a transaction.

96. What happens when a transaction is added?

The backend saves the transaction and updates the related account balance based on transaction type.

97. What happens when a transfer is added?

The source account balance decreases and the destination account balance increases.

98. What happens when a goal contribution is added?

For a deposit, account balance decreases and goal saved amount increases. For a withdrawal, account balance increases and goal saved amount decreases.

M. Testing Questions

99. How did you test the project?

The project has backend safety tests and frontend build checks. Manual testing is also done by running the app and checking major flows.

100. What automated tests are included?

Tests check secret safety, hidden reset tokens, frontend popup removal, statement import safety, pinned dependencies, offline chat topics, family removal, and ledger connection.

101. How do you test frontend build?

By running the frontend build command and checking that it compiles successfully.

102. How do you test the database?

By running the app, logging in, adding accounts, adding transactions, checking balances, and verifying records in the UI.

103. How do you test import?

By uploading sample bank statements, reviewing preview rows, confirming import, and checking if transactions appear correctly.

104. How do you test ledger?

By adding income, expense, and transfer transactions, then checking debit, credit, and running balance.

N. Error and Limitation Questions

105. What are the limitations of your project?

It does not use real bank API sync, scanned PDF OCR, mobile app support, or online large language model AI.

106. What happens if MySQL is not running?

The app cannot load database data until MySQL is started and configured.

107. What happens if dependencies are missing?

The app may fail to start. The user must install required Python and Node dependencies.

108. Why did pandas get removed from import processing?

Because some systems block heavy pandas or numpy DLL files. The project now uses lighter libraries for CSV, Excel, and PDF statement import.

109. Is the project scalable?

It is suitable for a local academic project. For many users in production, deployment, security, database indexing, monitoring, and cloud hosting would need improvement.

110. What is future scope?

Future scope includes bank API sync, OCR for scanned statements, mobile app, advanced forecasting, EMI calculations, and optional online AI integration.

O. Interview-Style Technical Questions

111. Explain the project architecture.

The architecture has three layers: React frontend, Flask backend, and MySQL database. The frontend sends API requests to Flask. Flask applies logic and reads or writes data in MySQL.

112. What is REST API?

REST API is a way for frontend and backend to communicate using HTTP methods such as GET, POST, PUT, and DELETE.

113. What is CRUD?

CRUD means Create, Read, Update, and Delete. The project uses CRUD for accounts, transactions, budgets, goals, and profile.

114. What is authentication?

Authentication checks whether the user is logged in and allowed to access the system.

115. What is authorization?

Authorization checks whether the logged-in user has permission to access a specific data record.

116. What is input validation?

Input validation checks user input before saving it, such as amount, date, account, and transaction type.

117. Why should amount be validated?

Because invalid or negative amounts can create wrong finance records.

118. Why should date be validated?

Because analytics and monthly reports depend on correct transaction dates.

119. Why should duplicate records be prevented?

Duplicate records make income, expense, balance, and analytics wrong.

120. What is the difference between frontend and backend?

Frontend is what the user sees in the browser. Backend handles data processing, login, finance logic, and database operations.

121. What is the difference between local storage and httpOnly cookies?

Local storage can be accessed by browser JavaScript. httpOnly cookies cannot be accessed by JavaScript, so they are safer for login sessions.

122. What is SQL injection?

SQL injection is an attack where unsafe user input is used to change a database query. Parameterized queries help prevent it.

123. How does your project reduce SQL injection risk?

The backend uses parameterized queries for database operations.

124. What is data normalization?

Data normalization means organizing data into related tables to reduce duplication and improve consistency.

125. Why use separate tables for accounts and transactions?

One account can have many transactions. Keeping them separate creates a clean one-to-many relationship.

P. Project Defense Questions

126. Your guide says the project is simple. How will you defend it?

The UI is simple for users, but internally it includes multiple finance modules, offline AI, Naive Bayes category prediction, ledger accounting, duplicate detection, analytics, budgets, goals, and secure login.

127. How will you prove algorithms are used?

I can show the category prediction, ledger formula, budget usage formula, savings rate formula, goal progress formula, duplicate detection, and month-wise analytics logic.

128. Why not use a complex AI model?

For a personal finance academic project, explainable local AI is better. It is private, offline, free, and easier to justify.

129. What is unique about your project?

It combines normal finance tracking with ledger accounting, statement import, duplicate detection, offline AI chat, and Naive Bayes category prediction.

130. What is the most important feature?

The most important feature is accurate money tracking through accounts, transactions, transfers, and ledger.

131. What is the hardest part of the project?

Handling different finance flows correctly, especially transfers, goal savings, statement import duplicates, and account balances.

132. What did you learn?

I learned frontend development, backend APIs, database design, authentication, finance logic, algorithms, testing, and user-friendly project design.

133. What would you improve next?

I would add OCR for scanned statements, bank API sync, mobile app support, better forecasting, and detailed credit card/loan calculations.

Q. Quick One-Line Answers

134. Is transfer income?

No. Transfer only moves money between own accounts.

135. Is saving to goal an expense?

No. It is tracked as savings because the user still owns the money.

136. Is credit card an asset?

No. Credit card balance is a liability.

137. Is loan an asset?

No. Loan is a liability.

138. Is bank account an asset?

Yes. Bank balance is an asset.

139. Is the AI online?

No. It is offline and local.

140. Is the project free to run?

Yes, after installing required local software.

141. What is the database?

MySQL.

142. What is the backend?

Flask.

143. What is the frontend?

React.

144. What is the category prediction algorithm?

Multinomial Naive Bayes.

145. What is the ledger formula?

Opening balance plus credits minus debits.

146. What is net flow?

Income minus expense.

147. What is budget usage?

Actual expense divided by budget limit, multiplied by 100.

148. What is goal progress?

Saved amount divided by target amount, multiplied by 100.

149. What is duplicate detection used for?

To stop repeated bank statement imports from creating duplicate transactions.

150. What is the final conclusion?

FinWise AI is a practical offline finance management system with explainable algorithms and useful personal money tracking features.

FinWise AI Demo Checklist

Use this order when showing the project to your guide.

1. Start the App

```
cd C:\FinWise_AI
.\run.cmd
```

Open:

`http://127.0.0.1:5000`

2. Login

- Show login page.
- Login with your test account.
- Mention that passwords are stored as hashes.

3. Dashboard

Show these cards:

- Total Money.
- Balance Money.
- Income.
- Expense.
- Savings.

Explain:

`Total Money = Balance Money + Goal Savings`

4. Accounts

Show account types:

- Bank.
- Cash.
- UPI Wallet.
- Credit Card.
- Loan.
- Investment.

Add or show two bank accounts, one credit card, and one loan account.

5. Transactions

Show three transaction types:

-
- Income.
 - Expense.
 - Transfer.

Explain that transfer is used for moving money from one own account to another own account.

6. Ledger

Open Ledger and explain:

$\text{Running Balance} = \text{Opening Balance} + \text{Credits} - \text{Debits}$

Show how income, expense, and transfer affect debit and credit.

7. Bank Statement Import

Open Transactions and show Import Bank Statement.

Explain:

- CSV, Excel, and text-based PDF are supported.
- Preview appears before saving.
- Duplicates are marked before import.

8. Analytics

Open Analytics.

Show:

- Month selector.
- Income.
- Expense.
- Net flow.
- Saved to goals.
- Top category.
- Category chart.

Explain that the page filters transactions by selected month.

9. Budget

Open Budget.

Show category-wise budget limit and spending usage.

Explain:

$\text{Budget Usage Percent} = \text{Actual Expense} / \text{Budget Limit} * 100$

10. Savings Goals

Open Savings Goals.

Show:

-
- Goal target.
 - Saved amount.
 - Still needed.
 - Add savings to goal.

Explain that goal saving is not counted as normal expense because the money is still owned by the user.

11. Finance Chat

Open Finance Chat and ask these questions:

What algorithms are used here?
Is this offline?
What is balance money?
How does transfer work?
Explain ledger.

Mention that the answers come from local rules and local finance data.

12. Smart Tools

Open Smart Tools.

Show auto entries for repeated income or expenses such as:

- Salary.
- Rent.
- EMI.
- Bills.
- Subscription.

13. Profile

Open Profile.

Show:

- Name.
- Email.
- Password change.
- Theme selection.
- Logout button.

14. Reports

Open Reports.

Show export or report options available in the app.

15. Final Explanation

End with this short summary:

FinWise AI is an offline personal finance management system. It tracks accounts, income, expenses, transfers

Screenshot List For Report

Take screenshots of:

- Login page.
- Dashboard.
- Accounts page.
- Transactions page.
- Statement import preview.
- Ledger page.
- Analytics page.
- Budget page.
- Savings goals page.
- Finance chat page.
- Profile page.